Mathe(ML)atics

from proofs to predictions

Vednashi Vaghela

Department of Mathematics
IISERB

October 17, 2025

Overview

1. Machine Learning - its definition, intuition, and types

2. Algorithm and Math of PCA - Unsupervised ML

3. Math of Linear Regression and Stochastic Gradient Decent (SGD) - Supervised ML

what is ML

Intuition of ML

Can be thought of as an algorithm that optimizes a function — usually minimizing a loss function (error) or maximizing a reward, in laymen terms - to find the best model parameters for a given task.

Formal definition of ML

Machine Learning is a field of study that gives computers the ability to learn from data without being explicitly programmed. Formally, a computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.
Mitchell 1997

Examples

A model learns to predict house prices based on features like area and number of rooms.

graph of the supervised example stated (can do a better example figure)

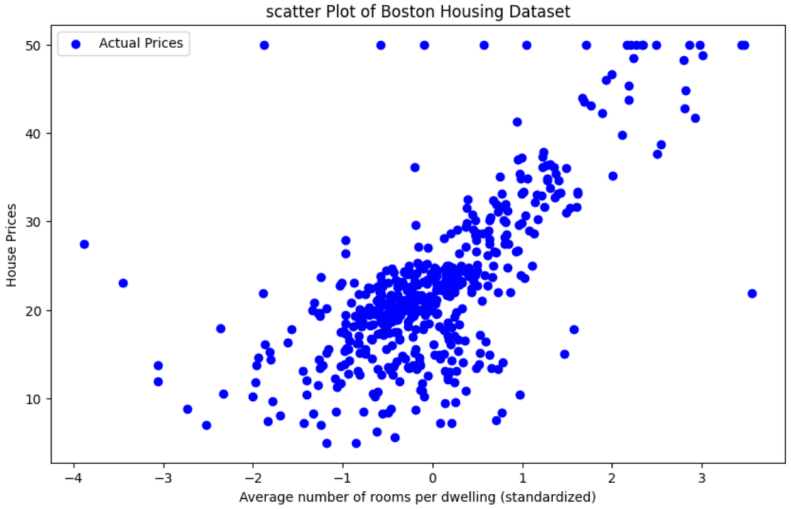
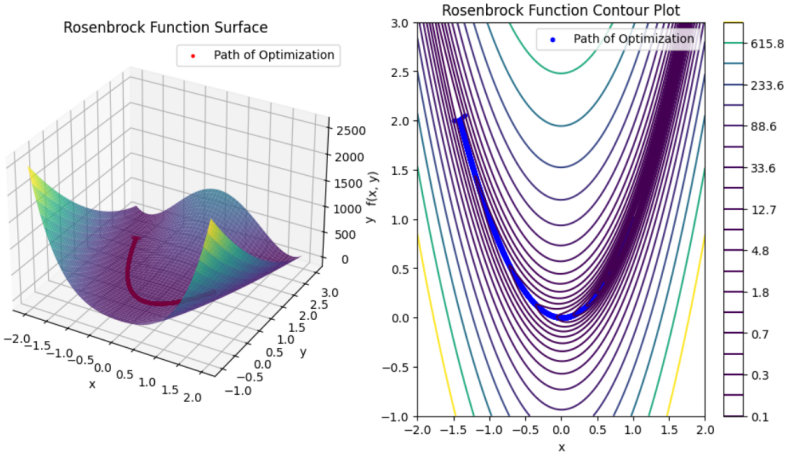


Figure: Boston housing scatter plot

plot of gradient descent over a surface



Why ML in the first place ??

We need Machine Learning because traditional numerical methods—like interpolation or differential equation solvers—work well when we have **explicit models or equations** describing the system. But in many real-world problems:

- **The underlying model is unknown or too complex** (e.g., natural language, images, human behavior).
- **Data is high-dimensional or unstructured**, making classical methods impractical.
- **Systems are dynamic or stochastic**, so static numerical methods can't adapt.

Machine Learning excels here by learning patterns directly from data and generalizing to new situations.

Types of ML

- **Supervised Learning:** A type of machine learning where the model learns from labeled data, using input-output pairs, to make predictions or classify new data.
- **Unsupervised Learning:** A type of machine learning where the model finds patterns, structures, or relationships in data without labeled outputs, such as clustering or dimensionality reduction.
- **Reinforcement Learning:** A type of machine learning where an agent learns to make decisions by interacting with an environment, receiving rewards or penalties, and optimizing its actions to achieve a goal.

Background of PCA (switch to board)

Principal Component Analysis (PCA): Principal Component Analysis (PCA) is a dimensionality reduction technique used to simplify high-dimensional data while preserving as much variance as possible. It is an **unsupervised ML technique**.
Uses:

- Reducing the number of features for faster computation.
- Visualizing high-dimensional data in 2D or 3D.
- Removing noise and redundancy.
- Pre-processing step for supervised learning tasks.

Math of PCA (board)

Eigenvalues in PCA

Let X be a data matrix$d \times n$ where d is the dimension of the data and n is the number of sample points and $\Sigma = \frac{1}{n} XX^\top$ its covariance matrix. Then Σ is symmetric, and all its eigenvalues exist, are real, and non-negative.

Reason: Since Σ is symmetric, the spectral theorem guarantees that all eigenvalues are real. Furthermore, for any vector v,

$$v^\top \Sigma v = v^\top \frac{1}{n} XX^\top v = \frac{1}{n} \|Xv\|^2 \geq 0,$$

so all eigenvalues are non-negative.

before we dive into the algorithm, here's **SOMETHING TO PLAY WITH**

Issues with PCA

1. **Time complexity:** Finding the eigenvectors & eigenvalues (the covariance matrix) has complexity typically $O(d^3)$, where d is the number of dimensions. *Example: Face recognition (eigenfaces).*
2. Data may not **necessarily lie in a low-dimensional linear subspace**.

Solution = kernel PCA – basically you map your data points to a higher-dimensional feature space using a kernel function, then perform PCA in that space. This allows capturing non-linear structures in the data.

Algorithm of PCA

1. **Center the data:**

$$x_i \to x_i - \bar{x}$$

2. **Compute covariance matrix:**

$$C = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^T$$

3. **Eigen decomposition:**

$$Cv_i = \lambda_i v_i$$

Sort λ_i in descending order and keep top k eigenvectors V_k.

4. **Project data:**

$$Z = XV_k$$

(Z = principal components)

Background of linear regression

The method of least squares was developed in the early 1800s by Carl Friedrich Gauss and Adrien-Marie Legendre for estimating celestial orbits. In the late 1800s to early 1900s, Francis Galton introduced *regression toward the mean* in studies of heredity. By the 20th century, regression became standard in statistics, economics, social sciences, and later machine learning.

Uses:

- **Prediction:** Estimating future values (e.g., sales forecasting).
- **Trend analysis:** Understanding relationships between variables (e.g., price vs. demand).
- **Risk assessment:** Evaluating factors affecting outcomes (e.g., medical or financial risk).
- **Feature importance:** Identifying which variables significantly impact the dependent variable.
- **Machine learning:** Serves as a baseline regression model.

Math of Linear regression (board)

Observations:

- Like PCA, w^* depends on a covariance "like" matrix, it also involves the labels y.
- Linear regression has a close form solution.

A few points to note

Questions

(i) Computational consideration? (as the direct calculation of w^* is very expensive)

(ii) Non-linear features present? (as the model proposed wont be able to catch non-linear feartures present)

(iii) What is the probabilistic view?

Questions answered (board)

(i) Gradient Descent Algorithm

(ii) Kernalizing the data and then fitting the model exactly how we saw in PCA

(iii) We care about this view, as it'll indirectly help us choose a cost function

before we dive into the algorithm, here's **a small SIMULATION** and **a PHeT SIMULATION**

Gradient Descent for Linear Regression

General Update Rule:

$$w_{t+1} = w_t - \alpha \nabla f(w_t)$$

For the cost function $f(w) = \frac{1}{2}\|Xw - y\|^2$:

- The gradient is $\nabla f(w) = X^T(Xw - y)$.
- The update rule becomes: $w_{t+1} = w_t - \alpha(X^T X w_t - X^T y)$.
- **Benefit:** This avoids calculating the expensive matrix inverse $(X^T X)^{-1}$ required by the closed-form solution.
- **Convergence:** It finds a local minimum. For convex problems like linear regression, this is also the global minimum.
- **Challenge:** For large datasets (large n), calculating the gradient over all data in each step is slow.
- **Solution:** Use **Stochastic Gradient Descent (SGD)**.

plot of optimization over a surface

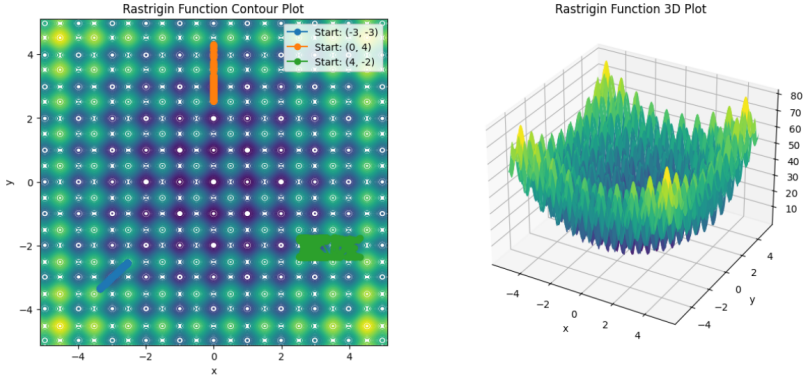


Figure: A very problematic surface

References

- GitHub Repository containing optimization code
- Scikit-learn Docs
- Github repo for other coding/ML related stuff

Thank You